



An edge-based IDS for the IoT using combined ML and generative AI models

Khaled Alanezi¹ · Tarun Annapareddy³ · Shafiullah Khan² · Shivakant Mishra³

Received: 1 May 2025 / Accepted: 19 November 2025 / Published online: 6 December 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

In this paper, the concepts of fog computing, machine learning and generative AI are utilized to construct an accurate and efficient intrusion detection system (IDS) for the Internet of Things. A hybrid approach in which the machine learning classifier for the IDS system is built using both real data and synthetic data is proposed. The real data used are collected at the edge of the network and deemed nonsensitive thereby preserving the security and privacy of the IoT environment. These real-time partial data are then used to train a generative AI model to produce synthetic samples that augment the minority class. We also implement principal component analysis (PCA) to minimize the system footprint of the developed models. To evaluate the proposal, we applied the approach to two datasets: an IoT traffic dataset (NSL-KDD) and an IoT telemetry dataset (ToN_IoT). The performance measures demonstrated a 75% reduction in the model loading time and a 58% reduction in the synthetic data generation time when PCA was applied to IoT traffic data. In addition, a classification accuracy of 86% was achieved for the IoT telemetry data using a random forest, while the precision, recall and F1 score attained for the minority-class labels improved, thus validating the effectiveness of the developed approach.

Keywords IDS · IoT · Generative AI · Generative Adversarial Networks (GANs) · Synthetic Data · Imbalanced Dataset

1 Introduction

The Internet of Things (IoT) is a revolutionary concept that refers to large networks of embedded devices that are often spread over wide geographical areas to serve important applications. IoT nodes are typically equipped with sensing, actuation, and networking capabilities, the latter of which allow them to communicate over the internet. An IoT application operates on top of this rich infrastructure to employ a mixture of device-to-device (D2D) and/or device-to-cloud communications to satisfy its design goals. A plethora of IoT applications utilize the IoT infrastructure to modernize human societies. For example, in health care [1], IoT devices can be wearable devices that are attached to the body of the patient or fabricated inside the patient's clothes. The sensors used in health care can be patient temperature sensors, blood sugar level sensors, blood pressure sensors, etc., while actuators can be insulin pumps or alerts related to the patient's condition. Education [2] is another area where the IoT can be used to achieve interactive and personalized learning. Examples of education devices include smart boards, cameras, and tablets for monitoring and interacting with students. Finally, another important field that the IoT

Thank you for trusting us with your document. Let us know how we did at <https://forms.office.com/r/53wL4caKpC?>

✉ Khaled Alanezi
khaled.alanezi@aasu.edu.kw
Tarun Annapareddy
tarun.annapareddy@colorado.edu
Shafiullah Khan
shafiullah.khan@aasu.edu.kw
Shivakant Mishra
mishras@cs.colorado.edu

- ¹ Computer Systems Engineering Program, College of Computer and Systems Engineering, Abdullah Al Salem University, Khaldiya, Kuwait
- ² Cybersecurity Engineering Program, College of Computer and Systems Engineering, Abdullah Al Salem University, Khaldiya, Kuwait
- ³ Computer Science Department, University of Colorado, Boulder, CO, USA

can serve is energy and sustainability [3], especially in areas with arid climates and limited freshwater resources such as the Middle East [4]. For example, IoT solutions for smart irrigation involve soil moisture, humidity, and temperature sensors that are used as inputs to intelligently control a drip irrigation system that acts as an actuator. In fact, governments and private organizations are now increasing the pace of their IoT deployments to streamline services in these sectors.

While an environment interwoven with the aforementioned computing capabilities can bring great benefits, it can also pose great risks, especially from cyber threats. The IoT can act as a playground for hackers seeking to harm national infrastructure or society. IoT applications are particularly vulnerable to cyber threats for several reasons. First, IoT devices are often deployed in remote areas with limited physical security. Second, these locations often result in intermittent connectivity, which can cause delays in the applications of security patches. Third, IoT devices typically have limited computing capabilities, making it difficult to run security procedures such as encryption and intrusion detection locally. Therefore, IoT solutions often face challenges in terms of privacy, access control, authorization, and verification. Typically, an IoT solution is a distributed application that spans four layers [5], each of which is subject to particular attacks. First, the perception layer is where the sensors are installed to read data from the environment, which can suffer from the injection of false data. On top of the perception layer, the network layer routes the data to edge or cloud servers for storage and processing. Afterward, the middleware layer can be subjected to man-in-the-middle (MITM) attacks, in which an attacker intercepts communication between system components. Finally, the application layer directly interacts with users, and it can suffer from data breaches or SQL injection attacks. To address these challenges, the design of a practical architecture that secures IoT systems while preserving privacy and satisfying IoT network performance requirements is presented in this paper.

Machine learning (ML) and artificial intelligence (AI) are becoming integral components of modern solutions for mitigating IoT security attacks. Utilizing ML, researchers train classifiers with labeled IoT data to classify sensory data or network packets as malicious or benign. Classifiers are also trained to detect the types of attacks [6] through an intrusion detection system (IDS). Although this approach is promising, it suffers from three major problems. First, publicly available coherent datasets that comprehensively cover IoT scenarios are scarce. Second, many available datasets exhibit label imbalances, where certain labels are overrepresented while others are underrepresented. Third, a privacy risk is incurred when exposing data to the cloud for training. Generative AI (Gen-AI) has been proposed to

address these issues [7]. With Gen-AI, a model is trained to capture patterns in the input data and can then generate completely new data points that resemble the original data. Gen-AI is capable of generating content, including text, music, images, and code. In a similar fashion, it can be used to generate network traffic for training IDS classifiers to detect threats in IoT networks, thus avoiding the need to share the original data. In this research paper, both ML and Gen-AI are used to take advantage of both approaches. The hypothesis is that IoT data vary in terms of their sensitivity. When an IoT environment experiences a sensitive context, the data must be kept local. In contrast, data generated during nonsensitive contexts can be offloaded to the cloud to train a generative adversarial network (GAN) model for data augmentation and synthetic data generation purposes. This improves the accuracy of intrusion detection while preserving the security and privacy of the IoT. To achieve this goal, we introduce an architecture that intelligently places the ML and Gen-AI models in the edge-fog-cloud continuum. The architecture takes the ability of each layer to host components into account while minimizing the exposure of users' IoT data by considering the sensitivity context. This architecture employs machine learning models with partial data augmented by synthetic GAN-generated data to improve the accuracy of intrusion detection. In summary, the contributions of this work are as follows.

- A practical architecture that combines ML and Gen-AI is developed to enhance the performance of an IDS within the edge-fog-cloud continuum while ensuring information privacy.
- The precision of this enhanced IDS is evaluated using various ML techniques based on two IoT datasets, namely, the NSL-KDD dataset, which represents network traffic data, and the ToN IoT dataset, which represents telemetry data. GAN models for data augmentation are applied to both datasets, and the accuracy of the system is measured accordingly.
- Measurements of the memory footprint, training time, and loading time of the system are performed to assess the practicality of the lightweight techniques used in restricted edge environments.

A key novelty of this work lies in building a practical architecture and validating its accuracy and feasibility using IoT datasets. The proposed architecture improves on the methods developed in previous studies by selectively placing components across the edge-fog-cloud continuum to preserve the privacy and security requirements of the IoT while meeting system performance demands.

The remaining sections of this paper are organized as follows. In Sect. 2, we review the related literature and compare

the different approaches proposed for building edge-based IDSs with our method. After that, in Sect. 3, we present the diagram of our architecture and describe its various components. The performance of the different techniques proposed in the architecture is presented in Sect. 4. Section 5 provides an analysis of the practicality of the work before we finally state the conclusion in Sect. 6.

2 Related work and article context

2.1 Benefits and challenges of edge-based IDSs

This related work section is focused on studies that employ edge technology to implement IDSs in IoT environments. Edge computing offers many benefits to IoT networks by placing computational resources such as processing power and storage closer to data sources, thereby reducing latency, mitigating intermittent connectivity issues, enhancing security, and lowering the financial costs that are often associated with relying solely on the cloud. A systematic review article published by Souza et al. [21] added two additional benefits. First, an IDS utilizing edge computing (or fog) is in a unique position for monitoring both internal and external network traffic. Second, its proximity to potential misbehaving IoT nodes enables faster incident responses to protect the network. It is essential here to differentiate between monitoring traffic on the edge and building and running models on the edge. The latter faces the unique challenge concerning the limited power and energy resources of IoT devices, which requires special handling, such as by building lightweight models or deploying collaborative architectures to split the training and/or detection tasks between the involved layers. An example of a collaborative structure is the work by Otoum [22], who proposed using volunteer computing

with nodes to assemble structures and aggregate nodes to a central head node. This node takes the responsibility of forwarding the network packets to a central server to construct the needed models from the accumulated data. In addition, IDS solutions face challenges related to data quality [7] in cyber physical systems (CPSs). First, data that can be generalized to different IoT systems are lacking because of the special nature of each deployment. Second, even if a dataset is available for use, it often suffers from imbalance issues where most of the records belong to the normal class. Third, the use of signature-based detection is not effective for the IoT because of the dynamic nature of such environments and the high frequency of emerging zero-day attacks. A survey conducted by Ahanger et al. [23] revealed that the use of AI can substantially benefit IDS solutions for the IoT, such as by enhancing their detection accuracy while allowing efficient computations to be performed on limited IoT devices. The survey highlighted that the solutions in the area fit within three categories, namely, ML, DL, and federated learning (FL).

2.2 General edge AI-based IDS solutions for the IoT

In Table 1, we list the recent works that have addressed the abovementioned challenges and compare them with our work. The comparison involves the techniques used to handle the encountered challenges and the evaluation criteria used. Mahadevappaa et al. [8] employed machine learning techniques to detect potential attacks. The focus of this work was on employing an incident response scheme to block misbehaving nodes while reducing their impact on the IoT network. The presented evaluation measured accuracy and simulated a cloud fog-IoT deployment scenario. Our work also looks at the performance of models in the IoT, fog, and cloud layers. Singh et al. [9] proposed combining

Table 1 Related edge-based IDS solutions

Authors	Employed Techniques				Evaluation		
	ML	DL	Gen-AI	Lightweight Models	Accuracy	Simulation	System Measurements
Mahadevappaa et al. [8]	X				X	X	X
Singh et al. [9]	X				X		X
Lee et al. [10]	X	X	X	X	X		
Wang et al. [11]	X		X		X	X	
Alweshah et al. [12]	X				X		
Alzubi et al. [13]	X	X			X		
Kalivani et al. [14]		X			X	X	X
Nie et al. [15]			X		X	X	
Sadaf et al. [16]	X	X			X		
Kathole et al. [17]		X			X		
Mazid et al. [18]		X		X	X		
Kirmani et al. [19]		X			X		X
Qiqieh et al. [20]		X			X		X
Our Work	X		X	X	X		X

an anomaly based IDS and a signature-based IDS with the aim of improving the resulting detection accuracy. Since this work focused on mobile edge computing (MEC) environments, its evaluation process involved performing system measurements by running the models on a Raspberry Pi 3 device, resembling an edge computing server. We also perform similar system measurements by looking at the memory footprints of the tested models, which is important for gauging the suitability of running models on limited devices. The work of Kalivani et al. [14] focused on the use of DL to build an accurate IDS using the CNN and LSTM deep learning techniques. They utilized the NSL-KDD dataset to evaluate the accuracy of their IDS, which is the same dataset that we use herein. The work by Kathole et al. [17] is another study that used deep learning to accurately perform intrusion detection in IoT networks. Their work used a deep ensemble method with the aim of combining the results obtained from multiple DL models to achieve enhanced accuracy. The accuracy improvement provided by their approach was demonstrated by applying the proposed technique to three datasets and employing various DL techniques, such as long short-term memory (LSTM) and a recurrent neural network (RNN). Finally, Sadaf et al. [16] introduced a DL-based IDS with the goal of performing binary classification on fog network packets (benign vs. malicious).

2.3 Generative and lightweight mechanisms for edge AI-based IDS solutions

Another work on edge-based IDSs is the study of Wang et al. [11]. They proposed using federated learning (FL) to construct an IDS while preventing IoT nodes from leaking data by using a GAN. Our work shares the same essence of employing GANs to create synthetic data for protecting the original data. Feature selection [12, 13] is another approach that has been introduced to create more accurate IDS models by monitoring only the relevant edge features. We use a similar technique but employ different measures of entropy, mutual information, and predictability, as discussed in Sect. 3.2. The use of GANs was also proposed in [15] as a three-phase approach. A feature selection step is followed by the construction of single attack detection models, which are subsequently combined to perform multiclass attack detection. A GAN was used in this work to implementing attack detection using the generator-discriminator model, whereas we use a GAN to generate synthetic models that are fed to an ML-based IDS model. Mazid et al. [18] used PCA to maintain the accuracy of intrusion detection while reducing the dimensionality of the input data for attaining improved system performance. Utilizing 1-D, 2-D and 3-D convolutional neural networks (CNNs), their developed models

exhibited good performance on two different IoT datasets when both binary and multiclass classifiers were employed. It is also important to cover works that focus on fog and edge computing, such as the work published by Alzubi et al. [13]. In this work, feature selection was used to address the high data dimensionality observed in edge-based IoT networks. This technique was used in conjunction with swarm optimization to achieve high detection accuracy. Lee et al. [10] also aimed to maximize the efficiency of running models on the edge by employing autoencoders (AEs) to reduce the number of features, thus creating a lightweight model that was capable of running on edge devices. We share a similar objective by using PCA to create our lightweight model.

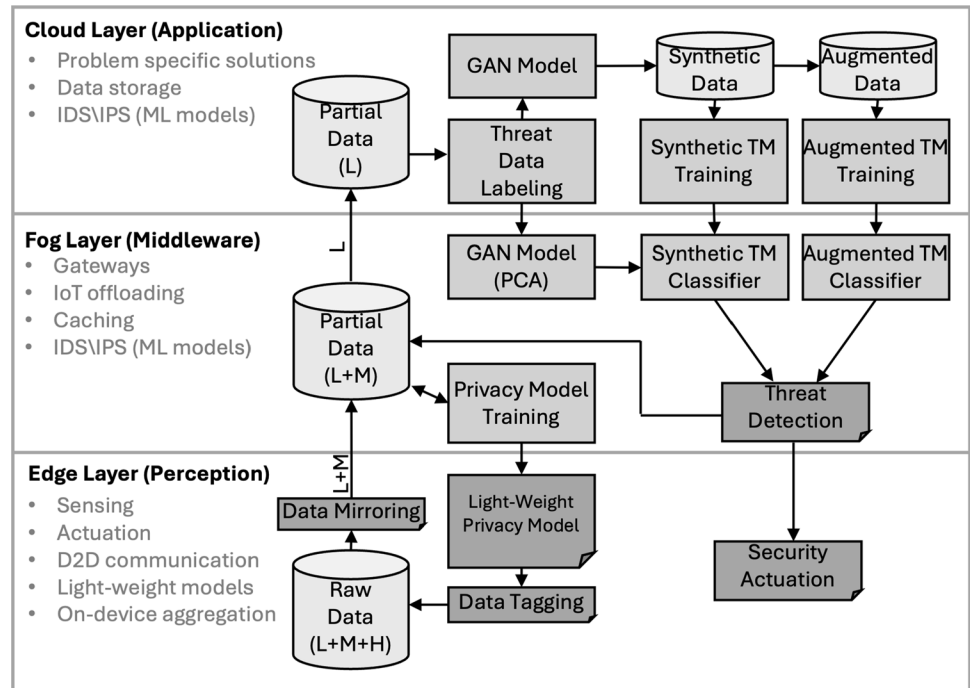
Although employing an IDS is essential, it can lead to privacy leaks. Hence, Mazid et al. [19] employed FL to construct an accurate IDS based on local models that share their parameters to construct a global model while preserving the privacy of the given data. Their experimental method demonstrated good accuracy in comparison with centralized learning (CL) approaches. In this work, system performance measurements were also covered, revealing improved communication overhead when the number of IoT devices was increased compared with that of CL. The selection of networks for achieving better performance was also conducted by Qiqieh et al. [20], who combined an artificial neural network (ANN) and the swarm algorithm. The experimental work in this paper covers three datasets to reflect on the top 20 identified features for improving the accuracy of IDSs.

3 Proposed architecture

3.1 Architecture general description

Utilizing Gen-AI in IoT applications requires overcoming many hurdles [24]. First, the memory challenge is caused by Gen-AI models having large footprints because of their vast numbers of parameters. In fact, training and loading standard Gen-AI models on resource-constrained edge devices is not feasible. Second, IoT applications often require real-time responses, particularly in IoT security scenarios, where attacks must be detected and mitigated promptly. Third, the energy challenge is induced by both the model training and execution processes incurring high energy consumption levels. In this section, we propose a novel architecture that addresses these challenges using distributed computing concepts. As shown in Fig. 1, our architecture spans the edge-fog-cloud continuum. Distributing the architecture across these three layers provides the flexibility to execute tasks on the layer that best satisfies their system requirements. At the bottom of the architecture,

Fig. 1 Class label distributions of the NSL-KDD dataset and augmented dataset



the edge-layer nodes run compressed ML models with small memory and energy footprints while satisfying strict latency requirements. In contrast, the fog layer consists of controller devices (often tethered) that are capable of training and running ML models. Fog-layer devices can load pretrained ML classifiers into memory to perform the required classifications in a timely manner. Finally, the top cloud layer is characterized by its abundant and shared resources. While this layer can host resource-intensive tasks such as the training and execution of Gen-AI models, data transfers over WAN links and the multitenant nature of the cloud often raise trust concerns. Hence, users often avoid using it to process highly sensitive data and prefer to keep such data on their premises at the edge. To fully understand the proposed architecture, we discuss the roles of devices in each of the three layers. First, in the **edge layer**, sensors, embedded devices, and microcontrollers perform data capture and initial filtering operations. Examples of devices in this layer include Arduino boards or Raspberry Pi devices. Second, the **fog layer** is composed of IoT gateways, cloudlet servers, mobile edge computing (MEC) servers, and local edge servers. Given their medium-tier capabilities, these devices can execute ML models to perform classification or support synthetic data generation tasks in a timely manner. Third, the **cloud layer** offers powerful computing services in the form of virtual machines (VMs) and containers. In addition, under the hardware-as-a-service (HaaS) model, the cloud provides CPUs, GPUs, memory, and storage resources. The architecture leverages cloud resources to perform Gen-AI

training, ML model training, or general architectural-level coordination tasks.

Other studies [25] have proposed the use of Gen-AI to support the development of machine learning models. In one extreme scenario, Gen-AI models generate synthetic data resembling the original data. These synthetic data can then be used to train models, thereby protecting the original data. At the other end of the extreme, Gen-AI models augment the original data, which often suffer from class imbalance issues, by generating additional minority-class samples. This in turn improves the accuracy of the utilized classifier while increasing the risk of exposing the original data. We adopt this concept in our architecture to enhance the level of data protection. Nonetheless, our architecture integrates supplementary contextual elements that are essential for safeguarding data. As depicted in Fig. 1, **raw data** originate at the edge layer. This layer functions as the trusted environment where initial interactions, such as sensing, actuation, and device-to-device communication, occur. In addition, a **lightweight sensitivity model** is used at this layer to protect the raw data. It can be considered the initial filter that determines which features should be non-negotiable in terms of sensitivity. In Sect. 3.2, we describe our lightweight mechanism for performing privacy classification. The features of the sensor value and/or network packet data records in this architecture can be tagged with three levels of sensitivity, namely, high, medium, and low, using the **data tagging** service. The idea here is that highly sensitive features should not leave the edge environment.

However, low- and medium-sensitivity features can be sent to the fog layer. Furthermore, only low-sensitivity features are sent to the cloud. **Data mirroring** performs data replication on the basis of this filtering approach. In the fog layer, the low- and medium-sensitivity features are used to perform **privacy model training**. The fog layer typically has components with medium-sized computing capabilities that work as gateways and/or run intrusion detection/prevention systems. These systems can utilize partial data to construct their models. The training process of the **privacy model** can use techniques in research to classify the sensitivity of data [26]. This record-based classification scheme can also be used to protect data records according to their context (see Sect. 3.2). In addition, medium-security records identified by the model can be identified such that only low-sensitivity records with low-sensitivity features are sent to the cloud.

Data reaching the cloud are subject to **threat data labeling** based on generic IDS models [27]. The resulting labeled records are then used to train the **GAN model** for generating **synthetic data**. Additionally, the GAN model is subjected to PCA (see Sect. 3.3) to reduce its memory footprint, and the produced lightweight **GAN model (PCA)** is then pushed to the edge for the generation of synthetic data near the data sources. Synthetic data can be used on their own or added to partial data to produce **augmented data**. Both are then used in the cloud to perform **synthetic threat model (TM) training** and **augmented TM training**. The produced classifiers are then pushed into the fog to perform the required **threat detection** work. On the basis of threat detection, **security action** can take place by blocking misbehaving nodes and notifying the network administrators. Finally, we evaluate the performance of these various concepts, such as the privacy model, PCA, and the threat model detection procedure, on the basis of the augmented data in Sect. 4.

3.2 Light-weight privacy classification techniques

Various techniques can be used to classify data into different sensitivity categories. A clear challenge is ensuring that the utilized method considers users' personal preferences, as different users perceive their privacy differently on the basis of their personal, cultural, and contextual values [28]. The work of [26] considered two factors when classifying information privacy, namely, universality and sensitivity. The former looks at how the public perceives the privacy of data, whereas the latter captures the amount of harm inflicted on users in data disclosure cases. Their work aligns with ours, as they classified data into three levels of sensitivity by using combinations of general labels and labels with high universality and severity. In addition, IoT sensor data can be subject to automatic privacy classification on the basis of the presence of certain content, such as faces in an image

[29]. Audio content can undergo privacy classification by providing special treatments for sensitive features that can aid in speech recognition or speaker identification [30]. We note here that these methods are orthogonal to our work, as they can all be utilized in the sensitivity model component of our system. Our architecture uses three measures for the classification of data privacy, with the goal of minimizing the features that reveal much information. First, mutual information (MI) [31] measures the dependency between a feature X and a target label Y . A high MI level means that a feature X reveals more information about a label Y according to the following formula:

$$I(X; Y) = \sum_{y \in Y} \sum_{x \in X} P_{(X,Y)}(x, y) \log \left(\frac{P_{(X,Y)}(x, y)}{P_X(x)P_Y(y)} \right) \quad (1)$$

Equation 1 [32] uses the joint probability mass function $P(X, Y)$ and the marginal probability of each variable $P(X)$ and $P(Y)$ to calculate the MI of X and Y .

The second measure we employ for privacy classification is entropy [33, 34]. Here, entropy measures the amount of uncertainty in a variable. The higher the entropy is, the more revealing a specific feature. Entropy can be measured using the following formula:

$$H(X) := - \sum_{x \in X} p(x) \log p(x) \quad (2)$$

In Eq. 2, $p(x)$ is the probability of a random variable X to represent the values of the corresponding feature [33].

The latter two measures are computationally lightweight and thus can be applied at the edge to eliminate high- and medium-sensitivity features. We note here that eliminating highly revealing features can reduce the resulting classification accuracy, but it is done for privacy reasons. Achieving an equilibrium between privacy and accuracy is a practical measure that can be revisited during system operations. Third, the architecture uses the measure of predictive power, which is assisted by the use of ML models such as naive Bayes or K-nearest neighbors (KNNs) and can be performed at the fog layer. High predictive power is indicative of a revealing feature and vice versa. To gain insights into the three measures, a weight is assigned for each component to combine the three aforementioned measures into one formula. The three measures are then applied to classify the most revealing features in the NSL-KDD dataset, as shown in Sect. 4.1.3.

3.3 Principal component analysis (PCA)

As our architecture employs edge and fog computing, it is important to minimize the sizes of models to be able to run

them on resource-limited IoT devices. PCA is a dimensionality reduction technique that aims to reduce the dimensions of a dataset while striving to retain the original classification accuracy. This in turn leads to a smaller model size, thereby leading to faster training and inference processes. We note that the gain derived from PCA not only reduces the memory footprints of models but can also improve their performance by minimizing the effect of the curse of dimensionality. We apply PCA to the NSL-KDD dataset before training the GAN model to produce a lightweight version. The GAN based on the original data and the GAN produced with PCA are then assessed on the basis of their system performance in Sect. 4.1.1. Our choice to apply PCA can be attributed to its low overhead, which makes it suitable for our edge-based design. However, a wide range of dimensionality reduction techniques can be considered for the same purpose [35]. When choosing among these techniques, it is important to balance the tradeoff between the computational overhead of a technique and its ability to capture the importance of evolving features due to dynamic environments. The justification for our choice of employing a GAN is the availability of the conditional tabular generative adversarial network (CTGAN) model [36], which is a GAN variant that was designed for generating synthetic data for single-table data.

3.4 Architectural adaptability

A key design goal of the proposed architecture is its flexibility to serve IDSs in diverse IoT environments. First, the architecture follows a layered approach that allows for the integration of different components or modules without impacting other layers. For example, in an extremely limited scenario, edge devices may support only lightweight computations. However, in a scenario with slightly better device capabilities, the edge device can perform ML inference tasks. In addition, the architecture is agnostic to the utilized dataset. In our evaluation, we use two datasets: NSL-KDD and ToN IoT. However, datasets from other domains and proprietary data captured by the IoT environment can also be used, provided that they are properly prepared and labeled. The architecture is also flexible in terms of addressing emerging threats through the interlayer coordination and continuous retraining of ML models to adapt to changing environments.

4 Performance evaluation

This paper follows an experimental approach to validate the proposed combination of ML and Gen-AI from two perspectives. First, a system-level validation is conducted by building

models and recording performance measurements such as memory size, loading time, and execution (i.e., data generation) time results. Our solution is distributed in nature and spans the edge-fog-cloud network. Owing to this distributed design, it is essential to perform system performance testing to ensure that the architecture delivers acceptable performance levels. Second, we evaluate the accuracy of IDS classifiers when incorporating the GAN-based synthetic data generation scheme utilizing partial data, thereby avoiding the need to share raw data and reducing the risk of exposing IoT security and privacy. This in turn requires the use of realistic IoT datasets with synthetic data generation and augmentation applied.

To ensure the credibility and generalizability of the results, we apply the two approaches mentioned above to two datasets representing network traffic data and IoT telemetry data, as covered in Sects. 4.1 and 4.2, respectively. This is essential, as attacks on IoT networks can emerge both at the network level and/or the device level [37]. The included datasets cover two main threat spectra faced by the IoT environment. On the one hand, NSL-KDD represents network-level attacks such as DDoS or MiTM attacks in which network packets are manipulated to attack the IoT network. On the other hand, the ToN IoT represents attacks at the data level, such as a data injection attack in which a compromised device produces erroneous data to harm the IoT network.

4.1 Evaluation on the IoT network traffic dataset

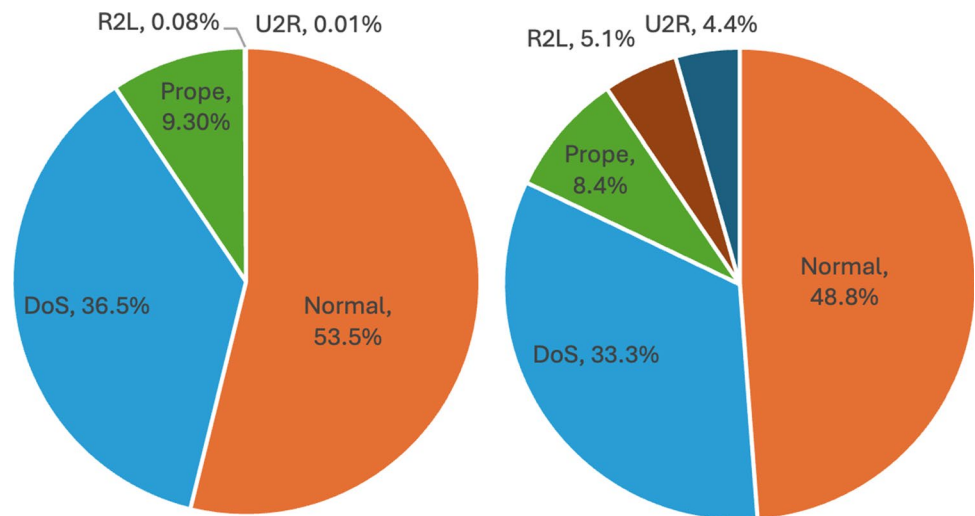
4.1.1 Dataset description

The NSL-KDD [38] dataset is an improvement over the KDD dataset in which duplicate records are removed and several enhancements are introduced, leading to a lower computational burden on machine learning models. Despite the dataset not being recent, the covered features and attack scenarios are still relevant. Explanations of the attack categories and examples from the dataset are shown in Table 2. Although the dataset has been described in many research papers [6] that have used it, we add the attack category type to the table and describe some attacks for ease of reference.

We also show the distribution of the attack labels that are present in the dataset in Fig. 2(a). We can see from the figure that despite the presence of an interesting and wide range of labels in the data, the labels belonging to R2L and U2R are severely underrepresented, with percentages of 0.08% and less than 0.01%, respectively. Synthetic data are employed in our architecture to address this issue and increase the accuracy of IDS models, as we will see in Sect. 4.1.2. The label distribution obtained after performing synthetic data generation is shown in Fig. 2(b). We study and compare the accuracies achieved for the two situations in Sect. 4.1.4.

Table 2 Descriptions of the attacks contained in the NSL-KDD dataset

Attack Class	Attack Explanation (selected examples)
DoS (denial-of-service): techniques that aim to overwhelm a system with illegitimate requests thereby preventing it from serving legitimate ones.	Land: Setting the source and destination information to the IP address in the TCP packet to the same value often leading to an infinite processing of the packet. Mailbomb: sending of massive number of emails to the mailbox or mail server thereby overwhelming its resources. Neptune: Sending flood of SYN messages with forged source address to overwhelm server resources while waiting for connections. Teardrop: an exploit for a bug in TCP in older operating system that prevents it from correctly processing fragmented packets.
R2L (remote to local): gaining illegal remote access through the network to a target node.	Ftp_write: misuse the anonymous FTP access feature to execute an exploit to be able to login as user FTP [39]. Guess_passwd: attempting to guess the user password by means of brute force. Imap: utilizing legacy email protocols to inject malicious content into an email message.
U2R (user to root): illegally gaining root access to a target by means of malicious software or back doors.	Multihop: Illegal access to a remote node by utilizing intermediary legitimate connections between the attacker and the victim. Rootkit: malicious software that causes an attacker to gain illegal root access to a machine. Buffer_overflow: an attacker writing extra data into memory buffer with the intention of corrupting the system or causing the machine to crash.
Probe: an attacker gather information about IPs and ports in the network to utilize them for future attacks.	Httpunnel: bypassing security measures such as firewalls and IDS by means of an HTTP tunnel to open a back door and become the root of the machine. Nmap: illegally gathering information about the network with the intention of causing harm using the network mapper tool. PortswEEP: scanning a target machine to discover open ports. Ipsweep: sending ping messages to a range of IP addresses to discover IP addresses of potential target machines in the network.

Fig. 2 Label distributions in the original and augmented NSL-KDD datasets

4.1.2 Performance of the model system

One of the main goals of our architecture presented in Sect. 3 is to utilize edge computing to process data locally near their sources. To achieve this goal, we first apply model optimization techniques that reduce the resource consumption level (memory and computations) while preserving high accuracy. We achieve this through principal component analysis (PCA), a technique that reduces the

dimensionality of data by producing a smaller set of features while preserving variance. This, in turn, produces a model that is smaller in size, thereby making the model less demanding in terms of computational and memory resources. To measure the effectiveness of our approach, we generate a minimized version of the dataset using PCA and train GAN models using both the original data and the PCA-minimized datasets. Table 3 provides a comparison between the two models.

Table 3 Comparison between the base model and the PCA-optimized model

Model	Features	Size (MB)	Training (hrs)	Loading (s)	Data Generation (ms)
GAN	42	4.3	12	0.16	700
GAN with PCA	21	2.2	7	0.04	290

Table 4 Feature privacy classification

Feature title	Feature description	Privacy score
src_bytes	Number of data bytes transferred from the source to the destination during the connection.	0.8699
count	Number of connections to the current destination measured in the last 2 seconds.	0.7741
dst_host_srv_count	Number of connections addressing the same port.	0.7502
dst_bytes	Number of data bytes transferred from the destination to the source during the connection.	0.7419
dst_host_same_srv_rate	Percentage of connections to the same service and to the same host.	0.6976
dst_host_diff_srv_rate	Percentage of connections to a different service and to the same host.	0.6949
same_srv_rate	Percentage of connections to the same service within the current destination in the last 2 seconds.	0.6918
diff_srv_rate	Percentage of connections to a different service within the current destination in the last 2 seconds.	0.6873
error_rate	Percentage of connections with errors within the current destination.	0.5668
flag_SF	A status indicating whether there is an error in the connection.	0.5635

We can see from the table that significant improvements are achieved in terms of various model performance measurements, reaching 75% in the case of the loading time and 58.57% in the case of the data generation time. The data generation time is the time elapsed between requesting the model to generate synthetic records and the time at which all records are received. These improvements are crucial since the model loading and data generation processes can, in principle, be requested in real time on the basis of the situation during runtime. The model loading time of the GAN with PCA is 40 ms, thereby making it suitable for loading during runtime whenever needed. However, the data generation time of 290 ms makes it more suitable for generating batch data for offline retraining. We also observe that the training time significantly decreases from 12 to 7 hours and that the memory footprint decreases by 48.85%. These results clearly justify our decision to optimize GAN models using PCA for providing enhanced usability at the edge.

4.1.3 Assessment of mutual information

As described in Sect. 3.2, an overall measure must be adopted to gauge the privacy-revealing scores of dataset features. The goal is to prevent a feature from being shared with a third-party server if it exceeds a defined threshold. The threshold may be defined by the user on the basis of system-specific experience. We combine the three measures of mutual information (MI), entropy, and predictive power (PP) using the following formula to derive a single value that can be compared with the threshold. These values can be adjusted based on user preferences:

$$\text{PrivacyScore} = (0.3 * MI) + (0.3 * Entropy) + (0.2 * NB_{pp}) + (0.2 * KNN_{pp}) \quad (3)$$

After applying the formula to the NSL-KDD dataset, we list the top 10 privacy-revealing features in the dataset along with a description of each feature taken from [40] in Table 4.

As shown in Table 4, the size of the data exchanged between the host and the destination in both directions (i.e., src_bytes and dst_bytes) clearly serves as a good indicator of attacks. In addition, the frequency of connections to the same port or destination is also indicative of attacks. This evaluation conducted based on the NSL-KDD dataset shows that the highly ranked features produced by our method align with those identified as important for implementing accurate IDSs [41], thereby proving the validity of the proposed method. Notably, for privacy labeling tasks, our architecture employs not privacy classification but rather a privacy scoring mechanism. This ensures a lightweight computation scheme that is suitable for execution on edge devices. This privacy scoring metric is reliable, as it combines three standard metrics: mutual information, entropy, and predictive power. Nevertheless, privacy remains a highly subjective measure that depends on the context, application, and user preferences. Therefore, the privacy scoring metrics and the sharing threshold should be continuously adjusted on the basis of system feedback regarding users' satisfaction with privacy-accuracy tradeoffs. Hence, our method serves as a pragmatic tool for quantifying privacy and adjusting system behaviors through a continuous feedback loop.

An important dimension to cover is the technical implementation of the privacy classifier. To test the combined ranking formula for the privacy scores, we apply it to the NSL-KDD dataset. Enforcing this formula in a real deployment scenario requires integrating it into the processing pipeline at the attribute level prior to transmitting data from the edge layer to the fog layer. Our attribute-based privacy mechanism aligns with other research on IoT privacy [42].

Table 5 Performance evaluation of logistic regression (NSL-KDD)

Logistic Regression	Base Model	Base Model Trained with Synthetic Data	Base Model Trained with Synthetic Data and PCA Applied
F1 Score	0.63	0.66	0.64
Accuracy	0.68	0.69	0.67
G-Mean	0.74	0.76	0.75

Table 6 Performance evaluation of the random forest (NSL-KDD)

Random Forest	Base Model	Base Model Trained with Synthetic Data	Base Model Trained with Synthetic Data and PCA Applied
F1 Score	0.71	0.74	0.71
Accuracy	0.76	0.77	0.72
G-Mean	0.79	0.81	0.78

Table 7 Performance evaluation of linear SVC (NSL-KDD)

Linear SVC	Base Model	Base Model Trained with Synthetic Data	Base Model Trained with Synthetic Data and PCA Applied
F1 Score	0.63	0.64	0.63
Accuracy	0.66	0.65	0.66
G-Mean	0.74	0.75	0.75

4.1.4 Model accuracy

We create nine machine learning models by varying the type of classification algorithm, the data used, and whether PCA is applied (refer to Fig. 2(b) for the label distribution after the synthetic data are applied). The accuracy measurements for these models, which are based on the F1 score, accuracy and the G-mean score, are reported in Tables 5, 6 and 7, respectively. Here, we include the latter two measures because of their importance for evaluating the accuracy of a model trained on imbalanced data. We start by describing the accuracy measure so that it can be used as a baseline for the other two measures. The formula for accuracy is shown below:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

Accuracy is a good measure when the employed dataset is balanced. However, it performs poorly in terms of spotting low-performance models when minority classes are identified. To solve this problem, the F1 score is introduced, and it has the following formula:

$$F1Score = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (5)$$

$$Precision = \frac{TP}{TP + FP} \quad (6)$$

$$Recall = \frac{TP}{TP + FN} \quad (7)$$

The F1 score provides a good measure for imbalanced datasets; however, it prioritizes positive classes. That is, a high

F1 score is achieved if the model performs well in terms of spotting true-positive and false-positive cases. This measure is used if the action performed on the basis of the positive class is costly (e.g., performing a life-threatening surgery or shutting down a critical system). Another measure that is used for imbalanced datasets is the G-mean, whose formula is as follows:

$$G - Mean = \sqrt{Sensitivity * Specificity} \quad (8)$$

$$Sensitivity = \frac{TP}{TP + FN} \quad (9)$$

$$Specificity = \frac{TN}{TN + FP} \quad (10)$$

The G-mean score is typically used when equal priority is given to spotting positive and negative classes.

We now describe the accuracy performance results of the different models. As shown in Table 5, when logistic regression (LR) was used, the results were low when both the F1 score and the accuracy measures were considered. The performance slightly improved when the synthetic data were added for all the cases. The G-mean score row follows the same trend of a slight improvement when synthetic data were added. However, it contains higher accuracy results, which means that the classifier did equally well for both positive and negative classes. As shown in Tables 6 and 7, compared with the linear support vector classifier (SVC) and the previously described LR classifier, the random forest classifier provided the best results. Here, again, we noticed slight improvements in all the measures when synthetic data were added. Finally, as shown in Table 7, when linear SVC was used, the performance was lower than that of the RF classifier, but the same trends were preserved. In

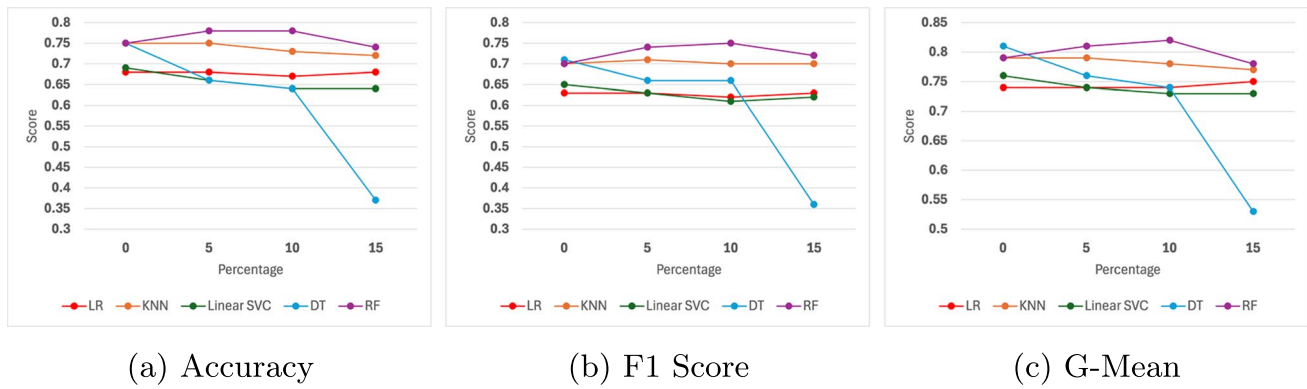


Fig. 3 Data augmentation accuracy measurements produced with normal data

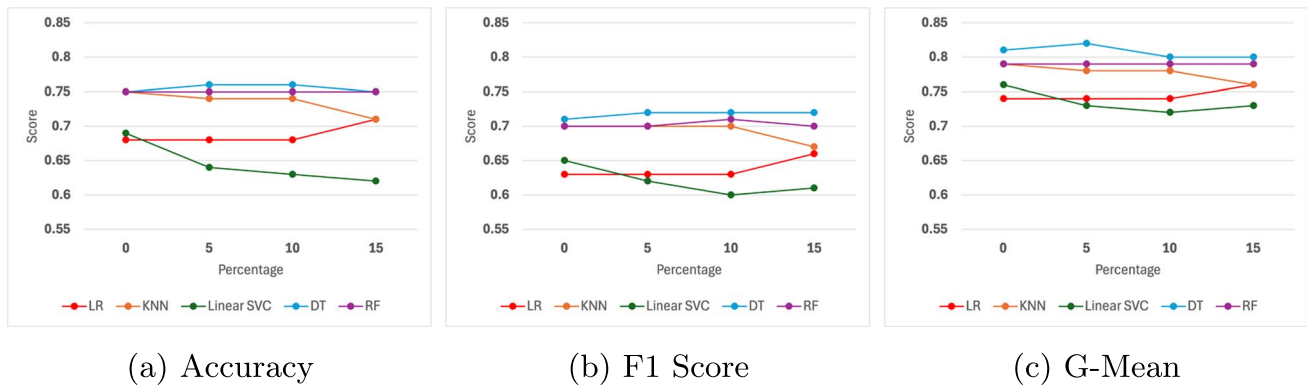


Fig. 4 Data augmentation accuracy measurements produced with PCA

general, three lessons can be derived from Tables 5, 6, and 7. First, the G-mean provided the best accuracy results for all the classifiers, which means that the models performed at the same level when classifying positive and negative cases. Second, the addition of synthetic data caused slight improvements in all the scenarios. Third, although the use of PCA provided significant system performance improvements in terms of the time and memory footprints of the models, it caused minor model accuracy degradations in all the scenarios.

4.1.5 Data augmentation percentage accuracy evaluation

An important factor when studying data augmentation is the percentage of synthetic data used and its impact on the resulting accuracy. To measure this aspect, we varied the share of synthetic data while building different ML models and measured the accuracy, F1 score, and G-mean. This exercise was conducted with synthetic data generated by a standard GAN (whose results are shown in Fig. 3) and a PCA-based GAN (whose results are shown in Fig. 4). The results of these experiments are available for verification in our public GitHub repositories [43, 44].

Looking at Fig. 3, which shows the models that utilized synthetic data based on normal GAN data, we see that adding synthetic data percentages of 5% and 10% increased all accuracy measurements for the RF classifier. However, increasing the percentage to 15% proved to be counter-productive for the same classifier. We also noticed slight improvements for the LR and KNN classifiers when 5% synthetic data were added. Increasing the percentage of data to 10% and 15% did not prove to be beneficial for the latter two ML models. An important observation concerns the DT, which suffered from any addition of synthetic data. This in turn shows that not all ML models can tolerate the addition of synthetic data, as this approach adds noise, which can impact various models differently.

We next investigated the impact of adding synthetic data generated from the PCA-based GAN model. As shown in Fig. 4, compared with the standard GAN case, the DT classifier did not exhibit the same performance degradation. These results can be attributed to the elimination of noise by PCA. Here, KNN and LR did not improve after synthetic data were added and did not experience a loss in accuracy. Finally, linear SVC suffered a slight performance degradation when synthetic data from both the standard GAN and the PCA-based GAN were added.

Table 8 Description of the TON IoT dataset (Fridge)

Described Aspect	Details of Description
Features in the dataset	Date: The date for logging the record. Time: The time for logging the record in HH:MM:ss format. Fridge Temperature: Exact temperature inside the fridge as measured in degrees Celsius by a temperature sensor connected to the IoT network Temperature Condition: Two-way classification of the fridge temperature as high or low with temperatures of 6 °C and above considered as high and below 6 °C considered as low. Label: Two-way classification of records as 0: normal or 1: attack. Type: 7-way classification with either normal or the specific attack type.
Labels in the dataset	Normal: A record collected under no attack scenario. Backdoor: Illegally bypassing the authentication of the IoT device by means of a hidden mechanism. DDoS: Hijacking the IoT device and using it to establish distributed attacks at scale aimed at other networks. Injection: Illegally executing commands on the IoT device to alter its behavior or steal data. Password: Attempts to gain access to the IoT device by compromising its authentication mechanism. Ransomware: Blocking access to the IoT device only to demand a ransom afterwards from the IoT owner. XSS: Launching a cross-site scripting attack to illegally gain control of the web-based admin panel of the IoT device.

Table 9 System performance of the TON IoT GAN model

Model	Features	Size (MB)	Training (minutes)	Loading (s)	Data Generation (ms)
ToN IoT GAN	5	0.90	3.5	0.02	146

4.2 Evaluation on the IoT-telemetry dataset

4.2.1 Dataset description

We used the TON IoT dataset [45] to evaluate the performance of our techniques on IoT telemetry data. The dataset covers a wide range of IoT scenarios representing telemetry (T), operating systems (O), and network (N)-level data. This dataset was developed in the laboratory to cover scenarios related to the 4.0 IoT industry standards and the Industrial Internet of Things (IIoT). Our focus was on the telemetry data, and data representing an IoT fridge were downloaded and used for experimentation. A detailed description of this dataset is given in Table 8.

4.2.2 Model system performance

The same synthetic data generation and augmentation schemes described in Sect. 4.1.2 were performed on the TON IoT dataset. We note here that PCA was not applied because the dataset contained a limited number of features, so we did not see any value in employing dimensionality reduction. To provide context, we compared the system performance results produced by the TON IoT dataset-based GAN model shown in Table 9 with the results in Table 3. It is clear that the small feature size (42 vs. 5) and number of records (12K vs. 39.9K) contributed to a significantly smaller model footprint. Compared with the NSL-KDD GAN model, the TON IoT GAN model was 80% smaller in size and generated synthetic data more than 4 times faster. We also noticed that the training time required on the TON

IoT dataset was on the order of minutes compared to that on the NSL-KDD dataset, which necessitated a training time of 12 hours. We note that the model loading time of 20 ms was small; however, the data generation time of 146 ms shows that the model can be used for offline data generation purposes to conduct retraining.

After training the GAN model on the minority classes of the TON IoT dataset, namely, the Ransomware and XSS records, we worked on generating 2000 records from each minority class and augmented the dataset with the 4000 new synthetic records. The label distributions of the TON IoT dataset before and after performing augmentation are shown in Fig. 5. The data augmentation process was aimed at reducing the share of the normal class from 37% to 34%, which was compensated by an increase in the minority-label classes to bring them to the same share as that of the other labels in the dataset. Our choice regarding the number of synthetic records was intended to investigate the impact of balancing this dataset on the accuracy of the ML intrusion detection models. Therefore, we trained the ML models on the original dataset and the augmented dataset using logistic regression and a random forest and report the results in Tables 10 and 11, respectively. Note that we did not include linear SVC here, as the algorithm is not suitable for low-dimensional and nonlinear data such as the IoT fridge data.

4.2.3 Model accuracy

We note from Tables 10 and 11 that there was a slight reduction in the overall accuracy of the models because of the addition of augmented data. This can be justified by looking

Fig. 5 Label distributions of the original and augmented TON IoT datasets

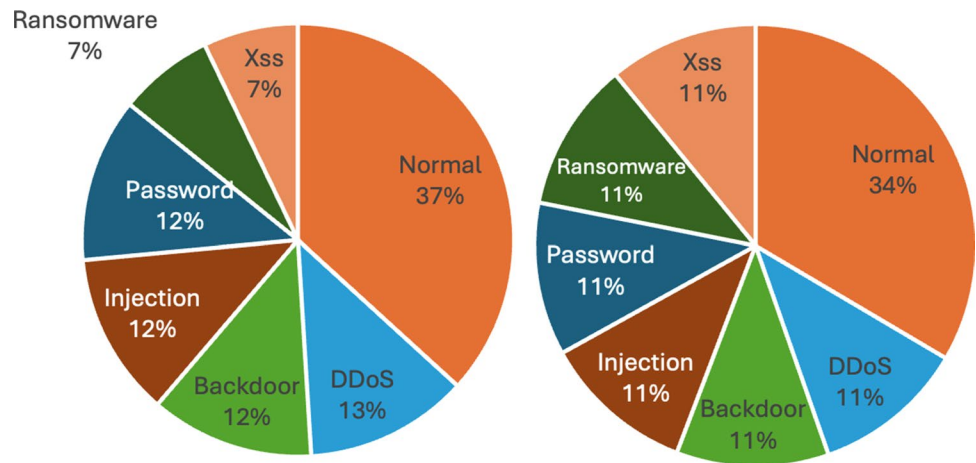


Table 10 Performance evaluation of logistic regression (TON IoT)

Logistic Regression	Base Model	Base Model Trained with Synthetic Data
F1 Score	0.66	0.65
Accuracy	0.65	0.64
G-Mean	0.80	0.79

Table 11 Performance evaluation of the random forest (TON IoT)

Random Forest	Base Model	Base Model Trained with Synthetic Data
F1 Score	0.86	0.85
Accuracy	0.86	0.85
G-Mean	0.92	0.91

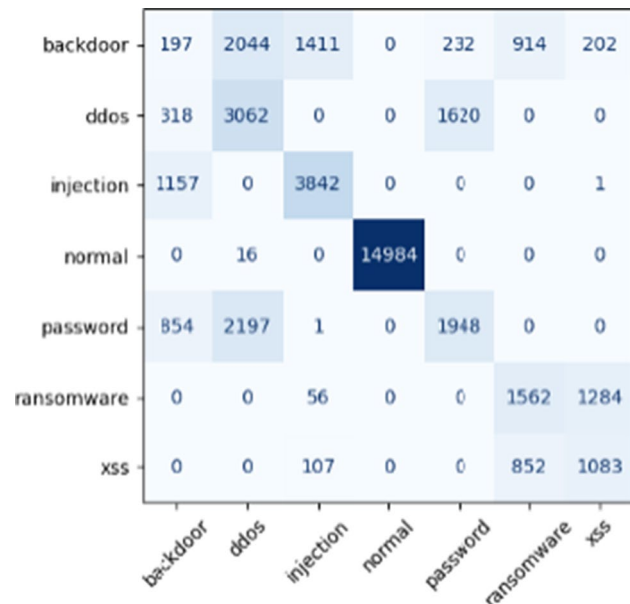


Fig. 6 Confusion matrix produced for logistic regression using the original data

at the confusion matrices produced by logistic regression, which are shown in Figs. 6 and 7. As shown in the figures, the accuracy achieved in terms of detecting majority classes such as the normal class decreased from 14984 to 14964 correctly labeled records. The same reduction in impact

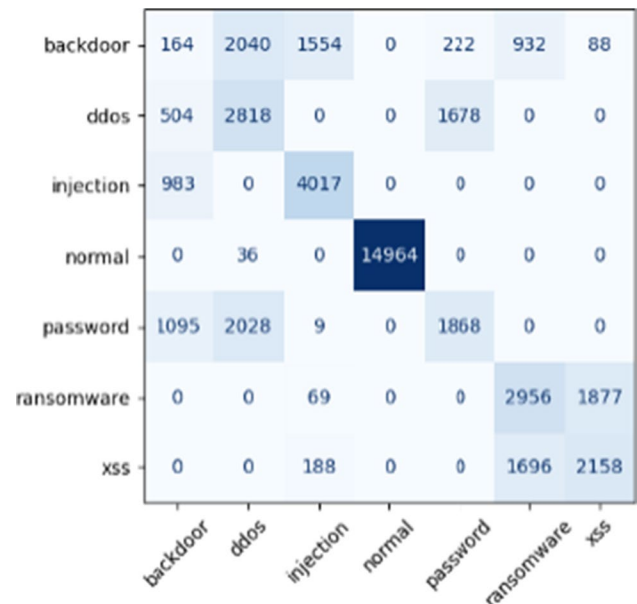


Fig. 7 Confusion matrix produced for logistic regression using the augmented data

occurred for the backdoor, DDoS, and password classes. However, an improvement was recorded for the injection class. An important measure here is to look at the accuracy attained for minority classes of ransomware and XSS since the added synthetic data corresponded to those classes.

When the accuracy of the confusion matrices was calculated, the ransomware detection accuracy improved from 53% to 60%, whereas for XSS, it remained the same at 53%. This finding shows that the addition of synthetic data might add some noise but can also improve the accuracy of minority-class labels.

We repeated the same accuracy analysis while looking at the confusion matrices produced for the random forest classifier in Figs. 8 and 9. Notice from the figures that the added noise did not impact the majority normal class at all. In addition, the reductions in the detection accuracies achieved for the other classes were minimal. Looking at the minority classes of ransomware and XSS, the former experienced a reduction from 77% to 74%, whereas the latter experienced an improvement from 60% to 72%.

Returning to the overall accuracy, we noticed that the performance of the random forest was superior to that of the logistic regression model, with an accuracy of 82% and a G-mean score of 92%, which are typical measures for imbalanced data such as ours. The same slight reduction in accuracy was observed here. While looking at the confusion matrices of Figs. 6 and 7, we see that the performance of both models remained good.

Finally, the takeaway message from looking at the accuracy results of the ML models built on the basis of the original dataset and comparing them with those built on the augmented datasets is that synthetic data generation can add noise to well fitted ML models, which can cause slight accuracy degradations. However, this can increase the accuracy achieved in terms of spotting minority-class records. This tradeoff is worth investigating on its own. In the next section, we investigate the other accuracy measures, precision, recall, and the F1 score.

4.2.4 Quantitative model accuracy measures

In this section, we performed a more detailed quantitative analysis by examining additional accuracy measures for the RF model. Note that the RF model was chosen because it achieved higher accuracy than the LR model did. In Table 12, the precision, recall and F1 score values produced for the original data and the augmented data are presented side

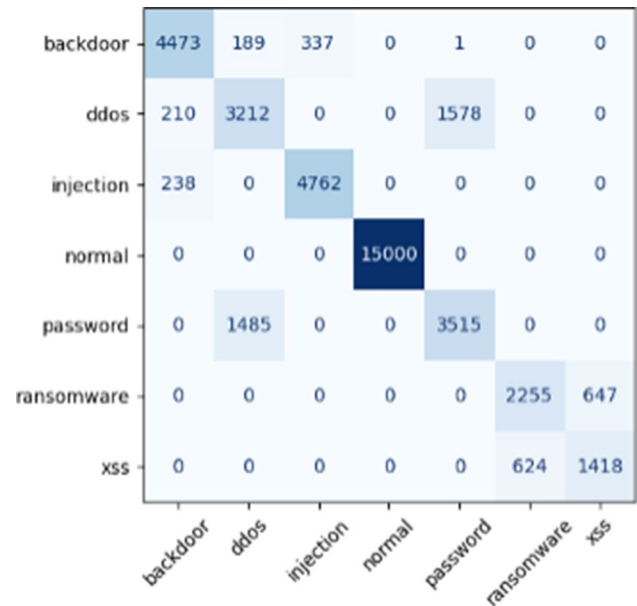


Fig. 8 Confusion matrix produced for the random forest using the original data

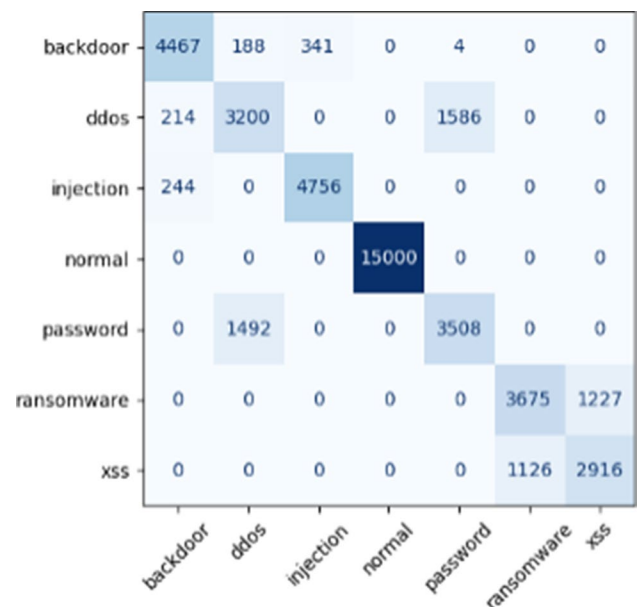


Fig. 9 Confusion matrix produced for the random forest using the augmented data

Table 12 Classification reports for random forest classifiers

Class	Original Data			Augmented Data		
	Precision	Recall	F1 Score	Precision	Recall	F1 Score
Backdoor	0.91	0.89	0.90	0.91	0.89	0.90
DDoS	0.66	0.64	0.65	0.66	0.64	0.65
Injection	0.93	0.95	0.94	0.93	0.95	0.94
Normal	1.00	1.00	1.00	1.00	1.00	1.00
Password	0.69	0.70	0.70	0.69	0.70	0.69
Ransomware	0.78	0.78	0.78	0.77	0.75	0.76
XSS	0.69	0.69	0.69	0.70	0.72	0.71

by side for comparison purposes. First, the results show that the backdoor, DDoS and injection classes maintained their detection accuracies across all the metrics despite the inclusion of augmented data. In addition, the normal class maintained its perfect detection accuracy. Second, for the minority classes of ransomware and XSS, the former experienced a slight accuracy degradation, whereas the latter showed improvements across all three measures. Such results motivate the choice to add augmented data when minority-class attacks are more costly or damaging than majority-class attacks, thereby justifying the tradeoff.

5 Discussion

It is important to note that training GAN models involves temporal and energy overhead, as shown by the performance evaluation presented in Sect. 4. However, the proposed model adopts a cloudlet scheme, where a tethered (i.e., powered) computing node with sufficient resources is positioned at the network edge to perform resource-intensive tasks close to the data sources. The cloudlet server hosts the GAN and ML models preloaded in memory for performing synthetic data generation and ML classification as needed. These models can also be pretrained on cloud servers and deployed on edge servers for runtime use. We plan to explore the techniques proposed in recent works for improving the efficiency Gen-AI energy use at the edge [46] in future work.

Another aspect to consider is scalability, which is critical given the need for IoT IDSs that can handle the large volumes of network traffic and data generated by IoT devices. We used PCA in our architecture, which contributed to minimizing the system footprints of the GAN and ML models, as shown in the performance results presented in Sect. 4. Despite this improved scalability, further enhancements can be achieved by applying techniques that reduce the size and inference time of the Gen-AI model, such as knowledge distillation and model pruning (see [47]). In addition to energy efficiency, stress testing the proposed architecture under varying IoT network loads, diverse device behaviors, and dynamic threat types is important. Performing simulations would allow the architecture to be evaluated under extreme conditions and facilitate the selective application of optimization techniques from the literature [48] for efficiently and scalably deploying AI at the edge.

We also note that our work employs recent techniques concerning edge intelligence, Gen-AI and edge-fog-cloud architectures to construct efficient IDSs for providing enhanced IoT security. Other works have explored the integration of intelligence into IoT devices for diverse

applications, incorporating features such as P2P communication [49] or edge server collaboration [50]. Our work shares the goals of these techniques, namely, maximizing the benefits of integrating these emerging technologies to support diverse IoT applications.

Before we conclude the discussion section, it is important to highlight the limitations of this study. First, this work aimed to construct a scalable IDS on the edge, which is an environment that faces challenges such as limited resources, intermittent connectivity, and stringent response time requirements. We measured system performance aspects such as the memory footprint and classification time. However, a real-world deployment is needed to provide more comprehensive feedback on the practicality of our method and potential areas for improvement.

Additionally, the utilized datasets, NSL-KDD and ToN_IoT, might not cover the wide range of attack spectra that are observed in modern IoT networks. Nevertheless, our approach is generic, and other datasets can be employed for training the classifier as they become available. Second, there are limitations related to Gen-AI and ML that should be acknowledged. With respect to Gen-AI, the utilized model may fail to continuously generate samples that match the real-world distribution as time progresses and new patterns emerge. Regarding ML, the model may suffer from overfitting when enriched with data for minority classes. These issues may lead to increased false positives, which is particularly unacceptable in sensitive environments, since they can trigger unnecessary and costly interventions. To address these aspects, continuous feedback, retraining, and model assessments are needed to ensure that the model keeps pace with the emerging threats. The third limitation is the applicability of the solution for obtaining comparable results under real-world settings. Here, the simulation assumptions approximated certain aspects of real-world deployments, such as intermittent connectivity in dynamic environments and an evolving threat landscape. This challenge can be mitigated by deploying the solution in real-world settings while incorporating continuous feedback and improvement. Fourth, operational complexity must also be considered. Although ML and Gen-AI models can improve through automatic feedback loops, user participation remains essential for evaluating and refining the overall user experience.

The development of the necessary tools that ensure the participation of users in privacy control through an improved user experience represents a research area [51] that is orthogonal to our work. Finally, IoT communication encryption is becoming more prevalent, and the proposed solution must be tested in real-world settings to evaluate its performance under such conditions.

6 Conclusion

This paper presents an architecture that spans the edge-fog-cloud continuum and integrates ML and Gen-AI models to develop an efficient IDS for the IoT. Lightweight privacy classification was incorporated to minimize the amount of data leaving the edge, thereby protecting the privacy of IoT users while IDS models were being constructed. In addition, PCA was applied to generate lightweight GAN models suited for the architecture. The system and accuracy measurements obtained for the various constructed models demonstrated both the benefits and practicality of the proposed approach. This work provides guidance for future research exploring Gen-AI-based data augmentation to produce privacy- and security-enhanced IDS solutions by applying the concept to IoT datasets and evaluating both the performance and accuracy of systems. In future work, we plan to implement the complete architecture and further experiment with variables affecting its accuracy to evaluate additional aspects of the proposed solution.

Author Contributions Khaled Alanezi: contributed to the ideas, experimental work and writing. Tarun Annareddy: contributed to the ideas, experimental work and writing. Shafiullah Khan: contributed to the ideas and writing. Shivakant Mishra: contributed to the ideas and writing.

Funding This research received no external funding.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

References

- Kashani MH, Madanipour M, Nikravan M, Asghari P, Mahdipour E (2021) A systematic review of iot in healthcare: applications, techniques, and trends. *J Netw Comput Appl* 192:103164
- Badshah A, Ghani A, Daud A, Jalal A, Bilal M, Crowcroft J (2023) Towards smart education through internet of things: a survey. *ACM Comput Surv* 56(2):1–33
- Khatua PK, Ramachandaramurthy VK, Kasinathan P, Yong JY, Pasupuleti J, Rajagopalan A (2020) Application and assessment of internet of things toward the sustainability of energy systems: challenges and issues. *Sustain Cities Soc* 53:101957
- Muniasamy A (2020) Machine learning for smart farming: a focus on desert agriculture. In: 2020 International Conference on Computing and Information Technology (ICCIT-1441). IEEE, pp 1–5
- Driss M, Hasan D, Boulila W, Ahmad J (2021) Microservices in iot security: current solutions, research challenges, and future directions. *Proc Comput Sci* 192:2385–2395
- Revathi S, Malathi A (2013) A detailed analysis on nsl-kdd dataset using various machine learning techniques for intrusion detection. *Int J Eng Res Technol (IJERT)* 2(12):1848–1853
- Shahriar MH, Haque NI, Rahman MA, Alonso M (2020) G-IDS: Generative adversarial networks assisted intrusion detection system. In: 2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC). IEEE, pp 376–385
- Mahadevappa P, Murugesan RK, Al-Amri R, Thabit R, Al-Ghushami AH, Alkawsi G (2024) A secure edge computing model using machine learning and ids to detect and isolate intruders. *MethodsX* 12:102597
- Singh A, Chatterjee K, Satapathy SC (2022) An edge based hybrid intrusion detection framework for mobile edge computing. *Complex Intell Syst* 8(5):3719–3746
- Lee SJ, Yoo PD, Asyari AT, Jhi Y, Chermak L, Yeun CY, Taha K (2020) Impact: Impersonation attack detection via edge computing using deep autoencoder and feature abstraction. *IEEE Access* 8:65520–65529
- Wang T, Tang T, Cai Z, Fang K, Tian J, Li J, Wang W, Xia F (2024) Federated learning-based information leakage risk detection for secure medical internet of things. *ACM Trans Internet Technol*
- Alweshah M, Hammouri A, Alkhalaileh S, Alzubi O (2023) Intrusion detection for the internet of things (iot) based on the emperor penguin colony optimization algorithm. *J Ambient Intell Humaniz Comput* 14(5):6349–6366
- Alzubi OA, Alzubi JA, Alazab M, Alrabea A, Awajan A, Qiqieh I (2022) Optimized machine learning-based intrusion detection system for fog and edge computing environment. *Electronics* 11(19):3007
- Kalaivani K, Chinnadurai M (2021) A hybrid deep learning intrusion detection model for fog computing environment. *Intell Autom Soft Comput* 30(1)
- Nie L, Wu Y, Wang X, Guo L, Wang G, Gao X, Li S (2021) Intrusion detection for secure social internet of things based on collaborative edge computing: a generative adversarial network-based approach. *IEEE Trans Comput Soc Syst* 9(1):134–145
- Sadaf K, Sultana J (2020) Intrusion detection based on auto-encoder and isolation forest in fog computing. *IEEE Access* 8:167059–167068
- Kathole AB, Vhatkar K, Dharmale G, Chiwhane S, Kimbahune VV, Goyal A (2024) A novel approach to iot security for intrusion detection system using ensemble network and heuristic-assisted feature fusion. *J Discrete Math Sci Cryptogr* 27(7):2207–2217
- Mazid A, Kirmani S, Abid M (2024) Enhanced intrusion detection framework for securing IoT network using principal component analysis and CNN. *Inf Secur J A Glob Perspect* 1–21
- Mazid A, Kirmani S, Manaullah, Yadav M (2025) FI-IDPP: A federated learning based intrusion detection approach with privacy preservation. *Transactions on Emerging Telecommunications Technologies* 36(1): 70039
- Alzubi OA, Alzubi JA, Qiqieh I, Al-Zoubi A (2025) An iot intrusion detection approach based on salp swarm and artificial neural network. *Int J Netw Manag* 35(1):2296
- Souza CA, Westphall CB, Machado RB, Loffi L, Westphall CM, Geronimo GA (2022) Intrusion detection and prevention in fog based IoT environments: a systematic literature review. *Comput Netw* 214:109154
- Otoun S, Kantarci B, Mouftah H (2021) A comparative study of ai-based intrusion detection techniques in critical infrastructures. *ACM Trans Internet Technol (TOIT)* 21(4):1–22
- Ahanger TA, Ullah I, Algamdi SA, Tariq U (2025) Machine learning-inspired intrusion detection system for IoT: security issues and future challenges. *Comput Electr Eng* 123:110265

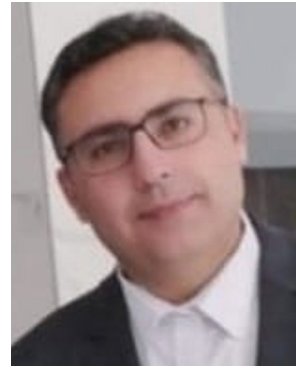
24. Wang X, Wan Z, Hekmati A, Zong M, Alam S, Zhang M, Krishnamachari B (2024) IoT in the era of generative AI: Vision and challenges. [arXiv:2401.01923](https://arxiv.org/abs/2401.01923)
25. Dwork C (2008) Differential privacy: a survey of results. In: International conference on theory and applications of models of computation. Springer, pp 1–19
26. Lu X, Li Q, Qu Z, Hui P (2014) Privacy information security classification study in internet of things. In: 2014 International conference on identification, information and knowledge in the internet of things. IEEE, pp 162–165
27. Da Costa KA, Papa JP, Lisboa CO, Munoz R, Albuquerque VHC (2019) Internet of things: A survey on machine learning-based intrusion detection approaches. *Comput Netw* 151:147–157
28. Westin AF (1968) Privacy and freedom. *Wash Lee Law Rev* 25(1):166
29. Zhao X, Lin S, Chen X, Ou C, Liao C (2020) Application of face image detection based on deep learning in privacy security of intelligent cloud platform. *Multim Tools Appl* 79(23):16707–16718
30. Chhagiani B, Gummeson J, Shenoy P (2024) Towards privacy-preserving audio classification systems. [arXiv:2404.18002](https://arxiv.org/abs/2404.18002)
31. Hoque N, Bhattacharyya DK, Kalita JK (2014) Mifs-nd: A mutual information-based feature selection method. *Expert Syst Appl* 41(14):6371–6385
32. Wikimedia Foundation I.: Mutual Information. https://en.wikipedia.org/wiki/Mutual_information
33. Wikimedia Foundation I.: Entropy (information Theory). [https://en.wikipedia.org/wiki/Entropy_\(information_theory\)](https://en.wikipedia.org/wiki/Entropy_(information_theory))
34. Luukka P (2011) Feature selection using fuzzy entropy measures with similarity classifier. *Expert Syst Appl* 38(4):4600–4607
35. Velliangiri S, Alagumuthukrishnan S et al (2019) A review of dimensionality reduction techniques for efficient computation. *Proc Comput Sci* 165:104–111
36. DataCebo I.: The Synthetic Data Vault Project (CTGAN). <https://github.com/sdv-dev/CTGAN>
37. Alanezi K, Mishra S (2024) An iot architecture leveraging digital twins: compromised node detection scenario. *IEEE Syst J* 18(2):1224–1235
38. Tavallaee M, Bagheri E, Lu W, Ghorbani AA (2009) A detailed analysis of the kdd cup 99 data set. In: 2009 IEEE Symposium on computational intelligence for security and defense applications. IEEE, pp 1–6
39. Radosavac S, Baras JS (2002) Detection and classification of network intrusions using hidden markov models. PhD thesis, University of Maryland, College Park
40. Dhanabal L, Shantharajah S (2015) A study on nsl-kdd dataset for intrusion detection system based on classification algorithms. *Int J Adv Res Comput Commun Eng* 4(6):446–452
41. Sinap V (2025) A novel hyperparameter tuning method for enhanced intrusion detection in network security. *Turkish J Eng* 9(3):519–534
42. Alpár G, Batina L, Batten L, Moonsamy V, Krasnova A, Guellier A, Natgunanathan I (2016) New directions in iot privacy using attribute-based authentication. In: Proceedings of the ACM International Conference on Computing Frontiers, pp 461–466
43. Annareddy T.: Experiments on Github by Tarun. <https://github.com/tarunannareddy/CTGAN/blob/main/NSL-KDD/classification-intrusion-detection.ipynb>
44. Alanezi K.: Experiments on Github by Khaled (NSL-KDD). <https://github.com/khaledalaneziq8/percentage-synthetic-data-accuracy/blob/main/CGAN-NSL-KDD.ipynb>
45. Alsaedi A, Moustafa N, Tari Z, Mahmood A, Anwar A (2020) Ton_iot telemetry dataset: a new generation dataset of iot and iiot for data-driven intrusion detection systems. *IEEE Access* 8:165130–165150
46. Wen J, Zhang R, Niyato D, Kang J, Du H, Zhang Y, Han Z (2024) Generative ai for low-carbon artificial intelligence of things with large language models. *IEEE Internet Things J* 8(1):82–91
47. Agrawal R, Kumar H, Lnu SR (2025) Efficient llms for edge devices: pruning, quantization, and distillation techniques. In: 2025 International Conference on Machine Learning and Autonomous Systems (ICMLAS). IEEE, pp 1413–1418
48. Surianarayanan C, Lawrence JJ, Chelliah PR, Prakash E, Hewage C (2023) A survey on optimization techniques for edge artificial intelligence (ai). *Sensors* 23(3):1279
49. Jo S, Lee J, Han J, Ghose S (2020) P2p computing for intelligence of things. *Peer-to-Peer Netw Appl* 13:575–578
50. Gou F, Wu J (2024) Optimization of edge server group collaboration architecture strategy in iot smart cities application. *Peer-to-Peer Netw Appl* 17(5):3110–3132
51. Jacobs D, McDaniel T (2022) A survey of user experience in usable security and privacy research. In: International conference on human-computer interaction. Springer, pp 154–172

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



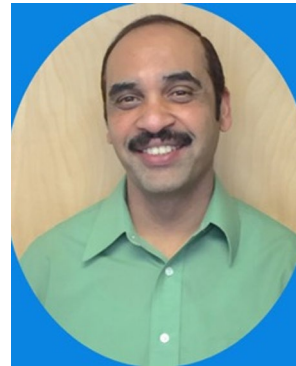
Khaled Alanazi is a researcher specializing in cybersecurity, IoT systems, and digital twins, with work spanning edge computing, smart environments, and AI-driven adaptive architectures. He also serves in a senior IT leadership role at Abdullah Al-Salem University, supporting major digital transformation initiatives alongside his academic and research activities.



Shafiullah Khan is currently serving as Professor of Cybersecurity Engineering, College of Computer and Systems Engineering, Abdullah Al Salem university Kuwait. His research focuses on wireless networks and security, cybersecurity, and related emerging technologies, where he continues to contribute through academic leadership, teaching, and scholarly work.



Tarun Annapareddy is a passionate software engineer with four years of experience building end-to-end data ingestion pipelines at Nutanix using distributed systems such as Apache Pulsar, Flink, Temporal, and Druid. He actively engages in open-source communities, writes articles on developer tools like Postgres, Flink, and Temporal, and has presented at industry events, including the Apache Pulsar Conference 2022.



Shivakant Mishra is a Professor in the Department of Computer Science at the University of Colorado Boulder, USA, with research interests in edge computing, cyber-safety, mobile and pervasive computing, and large-scale distributed systems. He has also joined the National Science Foundation (NSF), where he currently serves as a Program Director.